

UNIVERSIDADE DO MINHO
Departamento de Informática
UC: Processamento de Linguagens

Relatório do Trabalho Prático

Desenvolvimento de um Compilador de Fortran 77

Autores:

A107336 Alexandre Ferrete
A106872 Dinis Costa
A107333 Lucas Martins

17 de Maio de 2026

Conteúdo

1	Introdução	2
2	Opções de Implementação	2
2.1	Formato do Código Fonte	2
2.2	Estrutura do Projeto	2
2.3	Análise Léxica (<code>lexer.py</code>)	2
2.4	Análise Sintática e Árvore de Sintaxe Abstrata (<code>parser.py</code> e <code>ast_nodes.py</code>)	3
2.5	Análise Semântica (<code>semantic.py</code>)	3
2.6	Geração de Código (<code>bm_codegen.py</code>)	4
2.7	Subprogramas (<code>FUNCTIONS</code> e <code>SUBROUTINES</code>)	4
3	Otimização de Código	5
4	Gramática Utilizada	6
5	Testes	8
6	Fluxo de Execução do Compilador	8
7	Compilação e Utilização	10
8	Dificuldades de Implementação	10
9	Conclusão	10

1 Introdução

Este projeto consiste na implementação de raiz de um compilador de *Fortran 77* no âmbito das temáticas estudadas em Processamento de Linguagens. O compilador foi inteiramente desenvolvido na linguagem *Python*, utilizando ferramentas de suporte *PLY*. O código máquina gerado pelo programa tem como alvo a Máquina Virtual disponibilizada pela equipa docente. O projeto conta ainda com a fase de valorizações, incluindo otimizações e definição e chamada de subprogramas.

2 Opções de Implementação

2.1 Formato do Código Fonte

O grupo optou por suportar o **formato livre (free-form)**, em detrimento do formato de colunas fixas do Fortran 77 original. Esta decisão simplifica a especificação do analisador léxico e aproxima o ambiente de desenvolvimento das práticas modernas de escrita em Fortran, mantendo total compatibilidade com os exemplos do enunciado.

Os comentários são aceites em duas vertentes: linhas completas iniciadas pelos caracteres `C`, `c` ou `*` (seguidos obrigatoriamente por um espaço), e comentários *inline* delimitados pelo caractere `!`.

2.2 Estrutura do Projeto

```
1  src/
2      |-- __main__.py          # Ponto de entrada CLI do compilador
3      |-- lexer.py            # Analise lexica (PLY lex)
4      |-- parser.py           # Analise sintatica (PLY yacc)
5      |-- ast_nodes.py        # Definicao dos nos da AST
6      |-- semantic.py         # Analise semantica e verificacao de
7      tipos
8      |-- bm_codegen.py        # Gerador de codigo para a VM
9      |-- optimizer.py         # Otimizacoes ao nivel do Bytecode (01)
10     |-- ast_optimizer        # Otimizacao de fluxo (02)
11     |-- builtin_defs.py       # Suporte a funcoes intrinsicas de
12     Fortran
13 tests/
14     |-- hello.f
15     |-- fatorial.f
16     |-- ...
```

Listing 1: Estrutura de diretórios do projeto

2.3 Análise Léxica (lexer.py)

O analisador léxico é responsável por processar o fluxo de texto bruto do ficheiro de origem e decompô-lo numa sequência ordenada de *tokens*, descartando os elementos não necessários (espaços em branco e comentários). Utilizando o `ply.lex`, mapearam-se as palavras-chave da linguagem e os operadores fundamentais:

Categoria	Exemplos
Palavras-chave	PROGRAM, END, INTEGER, REAL, LOGICAL, IF, THEN, ELSE, DO, GOTO, CONTINUE, PRINT, READ, FUNCTION, SUBROUTINE, RETURN, CALL, ENDIF
Ops. Aritméticos	+, -, *, /, **
Ops. Relacionais	.EQ., .NE., .LT., .LE., .GT., .GE.
Ops. Lógicos	.AND., .OR., .NOT.
Literais Lógicos	.TRUE., .FALSE.
Identificadores	[a-zA-Z_] [a-zA-Z0-9_]*

A nível de precedência léxica, os operadores compostos como a potência (**) foram especificados em funções dedicadas no PLY para garantir a sua captura prioritária antes dos operadores simples como a multiplicação (*).

2.4 Análise Sintática e Árvore de Sintaxe Abstrata (parser.py e ast_nodes.py)

O *parser* valida se a sequência ordenada de *tokens* respeita a estrutura gramatical definida para este subconjunto de Fortran 77. Recorrendo ao `ply.yacc`, foi construído um analisador do tipo LALR(1) que traduz o programa estruturado numa Árvore de Sintaxe Abstrata (AST). Os nós da árvore foram divididos logicamente em duas famílias base:

- **Expressions:** Nós computacionais que processam e devolvem um tipo concreto de valor (ex: `BinOp`, `UnaryOp`, `Literal`).
- **Statements:** Nós imperativos que executam uma instrução ou alteram o fluxo de controlo do programa, sem produção de valor direto (ex: `Assignment`, `IfStmt`, `DoStmt`).

2.5 Análise Semântica (semantic.py)

O analisador semântico realiza a validação lógica do programa antes da geração de código. Este módulo constrói e gere uma Tabela de Símbolos estruturada para validar os diferentes escopos do programa.

As principais tarefas executadas por este componente são:

- **Validação de Declarações:** Garante que todas as variáveis foram declaradas antes de serem usadas e impede a criação de identificadores duplicados no mesmo escopo.
- **Verificação de Tipos:** Valida a compatibilidade de tipos em atribuições e operações binárias, rejeitando misturas inválidas (como operações aritméticas com `STRING`).
- **Validação de Fluxo (GOTO):** Monitoriza os desvios incondicionais, garantindo que todas as diretivas `GOTO` apontam para rótulos (*labels*) existentes e declarados.
- **Resolução de Ambiguidades:** Como o *parser* identifica inicialmente qualquer chamada com parênteses como uma `FunctionCall`, o semântico consulta a Tabela de Símbolos para processar e validar corretamente os casos que correspondem a acessos a vetores.

Normalização de Ciclos DO: Dado que no Fortran 77 os ciclos são delimitados de forma plana por rótulos numéricos, o analisador semântico implementa o método `_normalize_do_loops`. Esta rotina utiliza uma pilha de controlo interna para processar as instruções sequencialmente, agrupando de forma hierárquica todos os comandos pertencentes ao corpo do ciclo (`DoStmt`) até encontrar a etiqueta de terminação correspondente.

2.6 Geração de Código (`bm_codegen.py`)

O gerador de código atua no *backend* do compilador, sendo o responsável por percorrer a Árvore de Sintaxe Abstrata (AST) já validada e traduzi-la para a linguagem máquina linear da Máquina Virtual (VM). A sua tarefa principal é converter as estruturas abstratas criadas pelo *parser* (como expressões, atribuições e condições) numa sequência ordenada de instruções baseadas em pilhas, utilizando comandos nativos da VM como `start`, `stop`, `pushi`, `add`, entre outros.

Para que a execução seja correta, o gerador opera em duas frentes essenciais:

- **Separação de Contextos de Memória:** O gerador distingue automaticamente as variáveis que pertencem ao programa principal daquelas que pertencem a funções. Isto dita a emissão dinâmica de instruções globais (`pushg` e `storeg`) ou de instruções locais (`pushl` e `storel`), garantindo que os dados certos são lidos no espaço de memória correto.
- **Gestão de Operações Mistas:** Como a Máquina Virtual utiliza instruções aritméticas totalmente separadas para números inteiros (`add`, `sub`, `mul`) e números reais (`fadd`, `fsub`, `fmul`), o gerador monitoriza o tipo de dados dos operandos. Sempre que o código Fortran mistura inteiros com reais numa expressão, o gerador injeta de forma automática operadores de conversão (`itof` ou `ftoi`) na pilha antes de emitir a instrução matemática final, prevenindo falhas de execução na VM.

2.7 Subprogramas (FUNCTIONS e SUBROUTINES)

- **FUNCTIONS:** computam e devolvem um valor específico. e **SUBROUTINES:** blocos de ação isolados invocados pelo comando `CALL`).

Para evitar conflitos de nomes — como garantir que uma variável `I` dentro de uma função não altere acidentalmente o valor de uma variável `I` no programa principal —, o compilador isola completamente os ambientes através de uma ****Tabela de Símbolos Local**** dedicada a cada rotina. Esta gestão é coordenada pelo método auxiliar `_visit_routine`, que opera em três passos simples:

1. **Salvaguarda:** Guarda temporariamente a tabela de símbolos global ativa antes de entrar na rotina.
2. **Isolamento:** Inicializa uma tabela local vazia e exclusiva, registando apenas os parâmetros recebidos e as variáveis declaradas dentro daquela função ou subrotina.
3. **Restauração:** No fim da tradução da rotina, repõe a tabela global antiga para que o programa principal continue o seu fluxo sem interferências.

Para suportar o retorno de valores através do próprio nome da função (regra do Fortran 77), o compilador insere o nome da função na tabela local com o marcador `result: True`. Esta abordagem permitiu reutilizar a lógica comum de atribuição (`Assignment`) e armazenamento (`storel`).

3 Otimização de Código

O módulo de otimização do compilador atua em duas fases distintas: diretamente na Árvore de Sintaxe Abstrata (AST), realizando otimizações de alto nível, e na lista linear de instruções Assembly, aplicando otimizações de baixo nível. O objetivo principal é reduzir o tamanho do ficheiro final e o número de instruções executadas pela Máquina Virtual, mantendo o comportamento original do programa.

A fase da AST é gerida pela classe `ASTOptimizer`, que resolve operações aritméticas e lógicas entre constantes antes da geração de código. A fase seguinte é executada pela classe `Optimizer`, transformando e simplificando diretamente as instruções lineares do código máquina.

O sistema está estruturado em quatro níveis incrementais através da flag `-opt` (Níveis 0 a 3), cujas operações estão resumidas de forma compacta na Tabela 1.

Tabela 1: Níveis de Otimização e Respetivas Transformações

Nível	Designação	Operações Efetuadas
<code>-opt 0</code>	Sem Otimização	O código gerado é mantido intacto para facilitar a depuração.
<code>-opt 1</code>	Otimização Local	Dobragem de constantes (na AST e no Assembly), simplificação de operações neutras por técnicas de janela, remoção de código inalcançável após saltos e eliminação de rótulos mortos.
<code>-opt 2</code>	Otimização de Fluxo	Simplificação de desvios condicionais com valores constantes, eliminação de saltos redundantes para a linha seguinte (<i>fallthrough</i>) e limpeza de rótulos órfãos.
<code>-opt 3</code>	Otimização Agressiva	Execução cíclica e combinada das otimizações anteriores sobre as instruções até que não ocorram mais alterações.

Foram elaborados alguns gráficos ilustrativos das otimizações implementadas, mostrando assim o impacto das mesmas:

O comportamento dos testes face aos diferentes níveis de otimização, ilustrado na Figura 1, valida a arquitetura modular e incremental do sistema desenvolvido.

O teste `built_ins` demonstra um cenário estável: mantém-se invariável entre o nível `-opt 0` e `-opt 1` (74 instruções) devido à ausência de constantes puras elegíveis para dobragem. Contudo, o nível `-opt 2` consegue reduzir o código para 66 instruções. Este ganho deve-se à eliminação de rótulos mortos e saltos redundantes que tinham sido gerados de forma automática pelas rotinas de conversão e pelo cálculo do valor absoluto (`_emit_abs`). O ponto fixo é consolidado no nível `-opt 3` com 64 instruções, onde o pipeline cíclico remove as redundâncias remanescentes.

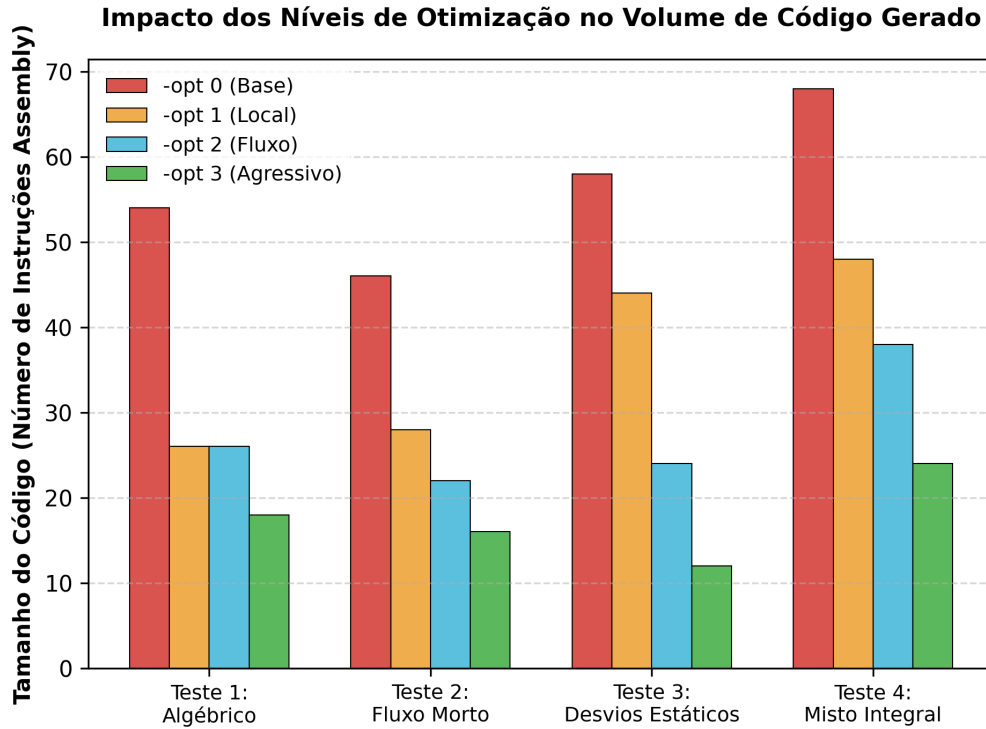


Figura 1: Impacto dos níveis de otimização no volume de código gerado.

Por outro lado, os testes `algorithm` e `flow` evidenciam uma flutuação nominal previsível: o volume de instruções no nível `-opt 2` (81 e 43 linhas, respetivamente) apresenta-se superior ao registado no nível `-opt 1` (71 e 27 linhas). Este fenómeno justifica-se pelo facto de o nível `-opt 2` isolar estritamente as otimizações de fluxo de controlo, omitindo por completo a dobragem algébrica de constantes aplicada no nível anterior. Em algoritmos com elevada densidade matemática, o ganho cumulativo de ambas as frentes de otimização só se manifesta na sua totalidade ao atingir o nível máximo de otimização agressiva (`-opt 3`), fixando os patamares mínimos absolutos de 71 e 20 instruções, respetivamente.

4 Gramática Utilizada

Abaixo apresenta-se a gramática BNF simplificada suportada pelo compilador:

```

1 program          ::= PROGRAM ID declarations_opt statements END
   function_defs_opt
2
3 function_defs_opt ::= function_def { function_def | subroutine_def
4                     }
5                     | <vazio>
6
7 (* Declaracoes *)
8 declarations_opt ::= { declaration }
9                     | <vazio>
10
11 declaration      ::= type declarator_list

```

```

11
12 type                ::= INTEGER | REAL | LOGICAL
13
14 declarator_list ::= declarator { COMMA declarator }
15
16 declarator        ::= ID
17                    | ID LPAREN NUMBER RPAREN
18
19 (* Instrucoes *)
20 statements        ::= statement { statement }
21
22 statement          ::= assignment
23                    | print_stmt
24                    | read_stmt
25                    | call_stmt
26                    | if_stmt
27                    | do_stmt
28                    | goto_stmt
29                    | label_stmt
30                    | continue_stmt
31                    | return_stmt
32
33 assignment         ::= varref EQUALS expression
34
35 print_stmt         ::= PRINT TIMES COMMA expression_list
36
37 read_stmt          ::= READ TIMES COMMA varref_list
38
39 varref_list        ::= varref { COMMA varref }
40
41 varref             ::= ID
42                    | ID LPAREN expression_list RPAREN
43
44 call_stmt          ::= CALL ID LPAREN [ expression_list ] RPAREN
45
46 goto_stmt          ::= GOTO NUMBER
47
48 label_stmt         ::= NUMBER statement
49
50 continue_stmt      ::= CONTINUE
51
52 return_stmt        ::= RETURN
53
54 (* Estruturas de Controlo *)
55 if_stmt            ::= IF LPAREN expression RPAREN THEN
56                    statements
57                    [ ELSE statements ]
58                    ENDIF | END
59
60 do_stmt            ::= DO NUMBER ID EQUALS expression COMMA expression
61                    | DO NUMBER ID EQUALS expression COMMA expression

```



```

62      COMMA expression
63
64  (* Subprogramas *)
65  function_def ::= type FUNCTION ID LPAREN param_list RPAREN
66                declarations_opt
67                statements
68                END
69
70  subroutine_def ::= SUBROUTINE ID LPAREN param_list RPAREN
71                  declarations_opt
72                  statements
73                  END
74
75  param_list ::= declarator_list | <vazio>
76
77  (* Expressões *)
78  expression_list ::= expression { COMMA expression }
79
80  expression ::= expression OP expression
81              | MINUS expression
82              | NOT expression
83              | LPAREN expression RPAREN
84              | ID
85              | ID LPAREN expression_list RPAREN
86              | literal
87
88  OP ::= + | - | * | / | **
89      | .EQ. | .NE. | .LT. | .LE. | .GT. | .GE.
90      | .AND. | .OR.
91
92  literal ::= NUMBER | STRING | .TRUE. | .FALSE.

```

Listing 2: Gramática BNF do Compilador

5 Testes

Para testar o programa, utilizaram-se os testes fornecidos pela equipa docente, dos quais não obtivemos o output correto no conversor. Apesar da equipa ter tentado, não se conseguiu chegar a uma correção do *output*, porém acreditamos, que se tivéssemos feito uma melhor gestão do tempo, teríamos sucedido. Além dos testes fornecidos, foram sendo adicionados testes complementares de modo a testar certos recursos que achamos serem necessários.

6 Fluxo de Execução do Compilador

O processo de compilação segue um modelo de *pipeline* sequencial, onde cada fase transforma o programa fonte numa representação progressivamente mais próxima da linguagem da Máquina Virtual. O fluxo de dados e a interação entre os módulos desenvolvidos estão sistematizados na Figura 2.

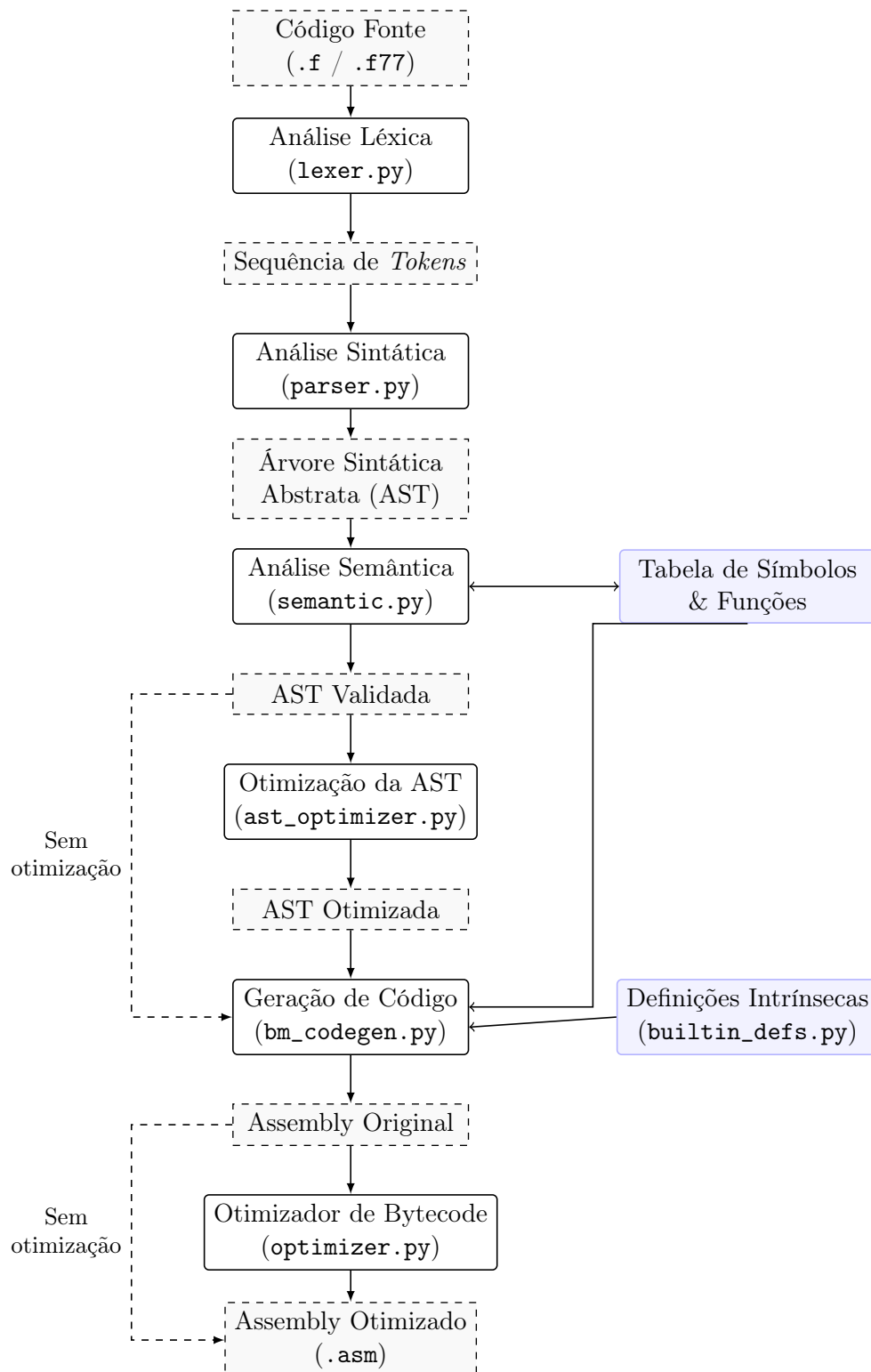


Figura 2: Fluxo de dados e arquitetura modular do compilador com dupla fase de otimização opcional.

7 Compilação e Utilização

Para compilar um ficheiro Fortran utilizando a interface de linha de comandos desenvolvida, utilize o seguinte formato de comando no terminal:

```
1 python src/__main__.py main <ficheiro.f> <nivel_opt> <saida.asm>
```

Onde <nivel_opt> corresponde a um valor numérico inteiro de 0 a 3.

8 Dificuldades de Implementação

Durante o desenvolvimento do compilador, o grupo enfrentou alguns desafios técnicos complexos relacionados com as restrições físicas e estruturais da Máquina Virtual (VM) de destino:

1. Ambiguidade entre acesso a array e chamada de função: Em Fortran, $A(5)$ pode ser tanto aceder ao índice 5 de um array como chamar uma função chamada A . O parser não consegue distinguir os dois casos só pela sintaxe, por isso trata ambos como chamada de função. A distinção só é feita mais tarde, no gerador de código, consultando a tabela de símbolos.
2. Estrutura dos ciclos DO: O Fortran não delimita os ciclos DO com chavetas nem palavras-chave de fecho — o ciclo termina quando aparece um número de linha específico noutra sítio do código. Isto significa que o parser gera o DoStmt sem body, e foi necessário implementar uma rotina no analisador semântico que percorre a lista de instruções e move as que pertencem ao ciclo para dentro do nó correcto.
3. O END fecha múltiplas coisas: A palavra END é usada para terminar o programa principal, funções, subrotinas, e também pode terminar um IF. Isto causou conflitos no parser que tiveram de ser resolvidos com regras específicas — por exemplo, criando uma regra endif que aceita tanto ENDIF como END.

9 Conclusão

O desenvolvimento deste projeto permitiu consolidar na vertente prática os conceitos fundamentais de engenharia de linguagens, traduzidos na implementação bem-sucedida de um pipeline completo de compilação para um subset expressivo da linguagem Fortran 77.

A resolução de problemas clássicos, tais como o desdobramento e normalização de laços de controlo não delimitados e o tratamento de assinaturas de chamadas ambíguas entre vetores e funções, forneceu um entendimento aprofundado acerca da abstração fornecida por linguagens de alto nível. Por fim, o desenvolvimento do sistema integrado de dupla fase de otimização provou o impacto direto da análise estática de código na economia de recursos de memória e na eficiência de processamento no ambiente da Máquina Virtual de destino.